ELSEVIER

European Journal of Operational Research 92 (1996) 573–590

EUROPEAN JOURNAL OF OPERATIONAL RESEARCH

Parallel processing for difficult combinatorial optimization problems

Catherine Roucairol *

PRiSM, University of Versailles-Saint-Quentin, 45 Avenue des Etats Unis, 78000 Versailles, France

Abstract

In identifying the general algorithmic problems most frequently encountered in designing and analyzing parallel algorithms (compatibility with machine architecture, choice of suitable shared or distributed data structures, compromise between communication and processing, and load balancing), we present recent research which has been carried out into parallelization of exact search methods such as Branch and Bound. We cover the main problems encountered with such a parallelization and present some theoretical and practical achievements in this field. The parallelization of heuristic search methods is shown to raise the same kind of issues.

Keywords: Combinatorial optimization; Parallel algorithms; Branch and Bound; Metaheuristics

1. Introduction

Parallel processing aims at solving larger problems in less time. The increasing number of processors in parallel machines now yields to massive parallelism allowing to cope with an increasing demand in memory space and processing time.

Parallelism has potentially important consequences for the area of Combinatorial Optimization. A large number of real-world problems in various fields (e.g. operations management: logistics, production scheduling and location and distribution problems, etc.; engineering: VLSI design and computer design; military OR: command and control systems, etc.) can be formulated as combinatorial optimization problems. Due to several factors (e.g. the internationalization of the activities of many firms, the need to produce solutions in real-time for some applications such as moving robots, speech recognition or job-shop scheduling) these problems are nowadays becoming increasingly complex.

The use of parallel machines makes it possible to find optimal solutions in less time and to increase the size of problems solved. This leads to substantial savings and will have considerable impact in many areas (VLSI circuit design, crew scheduling, vehicle routing, etc.).

However, proper algorithms have to be designed for these parallel machines as well as suitable development environments and programming tools (on these aspects see [85], [37] and [6]).

Work on parallel algorithms first started in the field of Numerical Computing. Based first on theoretical machine models such as PRAM, then on

* E-mail: Catherine.Roucairol@prism.uvsq.fr

PII S0377-2217(96)00009-4

experimental machines with rather 'exotic' architecture, and finally on commercial parallel or distributed multiprocessors computers, many parallel algorithms in the OR field were developed in the 80's.

In the 90's, three main factors have contributed to increase the number of experiments in the field of parallelization of algorithms:

– the wider access to large supercomputing networks (such as AHPCR-Minneapolis, CRPC-Rice or Nectar-Carnegie Mellon, etc. in the USA);

– an increasing number of research centers equipped with supercomputers (CCVR in France, CWI in the Netherlands, etc.);

– the success of transputer-based machines in Europe.

An exhaustive report on the research on parallel algorithms as listed in the first bibliographies [39,7,40,89] would now cover a whole book. In this paper we shall concentrate on the application of parallel algorithms to one field of Combinatorial Optimization showing how parallel computing can help fighting *against the combinatorial explosion* of the search space as the size of the problem increases.

Due to the theoretical complexity (NP complete class), the exact solution of classical combinatorial optimization problems, such as Travelling Salesman, Vehicle Routing, etc., necessitates the use of highly enumerative methods. Methods of the Branch and Bound (B & B) type search for the optimal solution by exploring potential solution trees which grow exponentially with the size of the problem. Heuristic search methods, like the Simulated Annealing or Tabu Search metaheuristics, find a 'good' solution which might not be optimal. They explore neighborhoods of solutions which may grow exponentially with the size of the problem.

In these situations the use of parallel algorithms is of great help.

This paper takes into account certain aspects of the B & B algorithms which have not been taken into consideration in recent surveys [32]. The aim of this paper is not to present another classification of parallel Branch and Bound algorithms but rather to point out certain algorithmic aspects encountered in parallelization. The various parallelization overheads due to load balancing, communication and sometimes useless exploration of certain nodes of the B & B tree will be presented. Parallel B & B algorithms found in the literature will be explored according to the solutions they propose to reduce one or many of these overheads. That helps the designer of a new parallel B & B algorithm to take advantage of the research already achieved in this domain. This paper presents also the historical evolution of the research in this domain which is tightly related to the evolution of the parallel computer architecture itself. We show that research has allowed to considerably widen the scope of real-life applications treated. Some of the most spectacular applications will be outlined. Lately, we showed that the parallelization of heuristic search methods may be affected by the same factors. This issue has not been treated in other surveys.

The paper is organized as follows. Section 1 shows the interest of parallel computing for solving difficult combinatorial optimization problems. Section 2 gives an algorithmic presentation describing the basic operations of B & B, points out the relation between the architecture of the multiprocessor machine used and the choices of parallel algorithms design. A simple classification of parallel Branch and Bound algorithms is retained. Section 3 emphasizes several aspects which must be taken into account when designing a parallel B & B: tree search overhead, well suited data structures to explore the search space, communication overhead, and load balancing. A list of references for applications is given in Section 4. Section 5 presents ideas for future research. Finally, Section 6 is concerned with the parallelization of heuristic search methods.

2. Parallelization of exact search space exploration methods

We will start by giving a general description of the Branch and Bound algorithm (Section 2.1). It is important to clearly identify the nature and scope of the work to be distributed among several processes. Then, we will show how machine and parallel algorithms are linked (Section 2.2) and how a simple classification into two types is sufficient to characterize parallel versions of B & B algorithms proposed in the literature (Section 2.3).

2.1. Sequential Branch and Bound

B & B is a search space exploration method, a technique frequently used in Operational Research (OR), Decision Sciences, and Artificial Intelligence (AI), to solve difficult combinatorial optimisation problems of the following type:

Given a finite set X, an objective function f defined on X, and a set of constraints defining a feasible set S included in X, find an element x in S such that $f(x) \leqslant f(y)$, *for all y in S.*

The area X to be explored is represented in OR by the problem subtree produced from an initial problem by means of recursive division (Branch and Bound, Branch and Cut procedures), in AI by a state transition graph (algorithm A*, AO*, etc.) or by a tree (game tree and α-β method, AND/OR tree in logic programming). A full examination is avoided thanks to the knowledge acquired along the exploration path, making it possible to exclude certain nodes on the graph or to prune certain branches from the tree.

Therefore, most of the work in a B & B algorithm consists of:

(i) *building the B & B tree:*

– *A branching scheme* constructs a B & B tree by recursively splitting X into smaller subsets, usually by partition, in order to end up with problems we know how to solve; the nodes of the B & B tree represent the subsets and the edges the relationship linking one parent-subset to its child-subsets created by branching.

– *An exploration strategy* selects one node among all the pending nodes (nodes not examined yet) in the tree according to priorities defined a priori by the application. Usually the priority $h(S_i)$ of a node S_i is either based on the depth of the node in the B & B tree which leads to 'depth first strategy', or on its presumed capacity for giving 'good' solutions leading to 'best first strategy'.

(ii) *pruning branches:*

– *A bounding function* v gives a lower bound for the value of the best solution belonging to each node or set S_i created by branching:

$$v(S_i) = \min\{f(x) \mid x \in S_i\}.$$

A node is 'evaluated' when a lower bound has been associated to it. This lower bound is used to define an exploration interval and, possibly, priorities among nodes to be explored like in 'best first' strategy where $h(S_i) = v(S_i)$.

– *An exploration interval* restricts the size of the tree to be built: only nodes whose evaluations belong to this interval are explored. The lower bound of this interval is the smallest value associated to a pending node in the current tree, the upper bound ub may be provided at the outset by a known feasible solution to the problem, or given by an approximate method. This interval is revised periodically. The upper bound ub is constantly updated every time a feasible solution is found during the expansion of the tree (value of the best solution known at that time, called the *incumbent*).

– *Dominance relationships* in certain applications may be established between solution subsets (S_i is dominant over S_j if the relationship $f(S_i) \leqslant f(S_j)$ is true for those subsets or one of their descendants) and will also lead to pruning non-dominant nodes.

From an algorithmic point of view, a B & B consists of carrying out a series of basic operations on a set of nodes of varying or the same priority: *deletemin* (select and delete the highest priority element), *insert* (insert a new element with predefined priority) and *delete greater* (delete elements with higher priority than a given value); see Table 1.

Table 1
Branch and Bound procedure

```
Procedure B & B (single least cost solution)
if a feasible solution x^0 is known then ub := f(x^0) else ub := ∞
d := makeheap (root)
do (d ≠ 0)
   x := deletemin(d)   / * select the node to expand with the
                         minimal priority h * /
   for each successor v of x do   / * expand x * /
      begin   / * updating the upper bound * /
      if (y is a feasible solution and (f(y) < ub)) then
         begin ub := f(y);
         deletegreater (ub, d)
         end
      endif
      if (y is not a leaf and (v(y) < ub)) then insert (y, d)
   endfor
enddo
```

The major difficulty is that the algorithm is working on an *irregular data structure*, created *dynamically* in the course of the algorithm, which is therefore not known at the outset. Furthermore, the number of elements, i.e. the size of the tree to be explored, grows exponentially with the size of the problems to be solved, and often remains too large: the program stops for lack of time or memory space before getting an optimal solution.

Therefore, parallelism is used to speed up the building of the tree by facilitating earlier more extensive pruning thanks to faster acquisition of knowledge.

2.2. Machines and algorithms

As we have already pointed out, although initial research from 1975 to the early 80's dealt with theoretical studies and simulated parallelism or experimental machines [82,35,28,14,52,29,83,65], in the following decade work developed both in trying to explain the surprising superlinear speed-up achieved [41,50,55,108,109,104,95], and in implementing parallelism on all sorts of machines as they came onto the market.

We will briefly describe how the parallel machines have emerged. The first non-experimental machines were mainly MIMD machines, with a small number of processors (not more than about twenty) but with a large processing capacity: the BBN Butterfly, the DEC VAX 11/782, the CDC Cyber 205 vectorial computer, the Alliant FX/8 and FX/2800, the Cray Research Cray X/MP, Cray2, and CrayY/MP, the Intel iPSC/1 and iPSC/2, the NCube NCube/10 and NCube/2 hypercube, the Sequent Balance Symmetry, the Encore Multimax, etc.

In Europe, Inmos built the T414 and T800 processors which have been used as basic units in many of the nearly emerged machines on the market such as TNode, MegaNode and the transputer-based networks. This last type is widely used in European university research laboratories.

The first massively parallel SIMD machines had several thousand processors with a fairly low processing capacity (1–4 bits): the Maspar MP-1 and the Thinking Machine Corp. CM-1 and CM-2.

Massively parallel machines have started emerging in the early 90's with systems having several thousand powerful processors (32 or even 64 bits) linked by high capacity networks: the IBM SP1, the Intel Paragon, the TMC CM-5, the Kendall Square Research KSR1, the Performance Computer Industries CS-2, the Archipel Volvox IS-860, the Maspar MP2, etc. All of these are distributed memory machines while the KSR1 is the only architecture with a virtual shared memory (see a detailed study in [6]).

From an algorithmic point of view, these different types of machines raise different problems. In SIMD machines, when the memory is distributed, several processing units have local or private memories and execute the same instructions synchronously, controlled by the same control unit (sequencer). The interconnection network transfers data between local memories. In MIMD machines, each processor executes asynchronously a sequence of instructions, which may be different from the others or not. SIMD machine calculations are fine-grained and compromises between processing and communication are very different from those encountered in MIMD machines. Load balancing must be carried out globally for the first type whereas it may be applied to a subset of processors in the second type, leaving the others executing their tasks.

One fundamental difference between these architectures depends on whether or not there is a shared memory. When there is one, the processors work by sharing global memory data: each processor reads the required data in the memory, processes it and writes the results to the memory. In machines with distributed memory, the processors work by exchanging messages: each processor reads data from other processors on one or more communication channels, processes it, then transfers the results to the processors which request them. The best-known interconnection networks are the ring and the hypercube. The time taken for a series of tasks no longer depends only on the time taken for each task but also of the position in the network of the processor executing the task. This does not imply that distributed algorithms are still strictly deterministic, although positions of processors in the network do not change from one algorithm execution to another. We must recall that factors such as message propagation delays, execution times on individual processors, etc., can vary from one execution to another,

imparting a non-deterministic character to parallel algorithms.

2.3. B & B parallelizations

The vast majority of parallelizations consists of sharing the work required to build the B & B tree and to explore its nodes simultaneously (branching and evaluation). The computation of the lower bound of a node, usually calculated quickly by a polynomial algorithm, is rarely shared (it may be required to speed up the initial work generation at the root of the tree [79]).

For greater clarity, we will retain a simple classification with two main types of parallelization (other classifications are examined in [32]) and with the gradual introduction of the various different parameters. In this way, a limited (although it may be very large) number of processes shares in exploring the tree.

In the first type (*distributed* or 'vertical'), each process is of the same type and must explore the subtree corresponding to one or more nodes initially allocated, independently of the other processes. The only information exchanged between processes is the value of a feasible solution.

In asynchronous algorithms, this value is transmitted to all the other processes as soon as it has been found. It may become a potential incumbent (best known value solution). Consequently it updates the known value of a process and hence locally reduces the exploration interval of this process. As a result this could lead to the elimination of a certain number of active nodes (requiring exploration) in the set managed by each process.

In synchronous algorithms, transmission takes place at fixed intervals (clock time [75]) or is set off when a specific number of iterations has been reached, or alternatively, when one of the processes is inactive for lack of work [74].

The major difficulty is due to the fact that the nodes to be explored are shared dynamically among several processes as exploration of the tree progresses. This raises the problems of load balancing and suitable data structures. How can a process which has finished obtain work rapidly from the others? Must a process always answer a work request? Even if it has 'almost' finished? How should it choose from several requests? How much work should it give to the others?

A wide variety of solutions have been proposed for regulating the workload: blackboard for shared memory machines, and static or dynamic strategy (details are given in Section 3.4). Since little communication is needed, this type of algorithm is well-suited for weakly-coupled multiprocessor architectures (which may have small local memories and a slow communication network). This provides a compromise between process grain (represented here by a sizeable number of nodes to be explored – a subtree) and relatively infrequent communication time. It is also suited to problems where search space exploration is carried out with a 'depth first' strategy (game tree, priority search for feasible solutions as in Integer Programming), and problems where we think that the incumbent will rarely be challenged.

In the second type of algorithms (*centralized* or 'horizontal'), a specialized central process manages the work (set of active nodes), updates the value of the incumbent, distributes the work (a small number NS of nodes, decided a priori) to each of the other processes. From the nodes they have been given, 'general purpose' processes develop a subtree, with only one or two levels. The result of their exploration (new nodes, possibly a new feasible solution) is then transmitted to the central process. The work to be done (work pool) is stored in a central process file and may also be stored locally.

The task is therefore less fine-grained. The main advantage of this algorithm is that the exploring interval is constantly and rapidly updated by the central process, making it possible to prune a large number of branches.

To ensure that work is carried out on 'good' quality nodes, this type of algorithm often uses a 'best first' strategy.

The processes are asynchronous but a few very limited experiments, with rather inconclusive results, use synchronization by making all the processes communicate as soon as they have all finished exploring one node [84,23] or NS nodes [65,33]. Waiting for the slowest process at these synchronization points leads to oversampling (unnecessary exploration of nodes on the B & B tree) which could have been avoided by immediate communication, e.g. updating the incumbent.

On the other hand, a fundamental problem is raised by asynchronous communications which are no longer between general purpose processes but between the central process and each of the other general purpose processes. This bottleneck is likely to cause a traffic jam in communications with the specialized central process especially towards the end of the algorithm.

An answer, as we will see later, consists to use several central processes which manages their own subset of general-purpose processes. In this case, several organisation schemes of the central processes are possible: ring, tree, etc. [79,27].

This type of algorithm has been widely implemented since experiments started in this field [82,14,92,104,94]. It is naturally best suited to machines with a small number of processors and an efficient transmission mechanism. This applies in particular to strongly-coupled multiprocessors such as asynchronous parallel machines with shared memories (the specialized process acts as a shared memory manager) but it has also been widely used on Hypercubes [28,29,23], transputer networks [84,57], in master–slave processor organizations, and in simulating shared memory on a network of workstations [15].

3. Parallelization overheads

On the one hand, the implementation of a parallel algorithm leads (or may lead) to processes carrying out a certain number of tasks which are not required in sequential processing, such as sharing and balancing the workload or transmitting information between processes, and on the other hand, tasks which are redundant or unnecessary for the work to be carried out. All these tasks will give rise to the various *balancing*, *communication* and *research* overheads which must be analysed in the design phase so that they are taken into account in the implementation. We shall explore these in further details in the following sections.

3.1. Research overhead

The success of parallelization should be measured experimentally in terms of absolute 'speed up'. It is the ratio of the time taken by the best sequential algorithm over that obtained with a parallel algorithm using p processors, for one instance of a problem. It is often defined, for simplicity, as the ratio of the time taken by a sequential algorithm implemented on one processor over the time obtained by the same algorithm parallelized and implemented on p processors (relative speed-up).

Efficiency reflects the effective use of resources provided by the processors. It is described in terms of the ratio of speed up over the number p of processors used.

For a B & B algorithm and its parallelization, the relative speed-up may sometimes be quite spectacular (greater than p), and at other times a total or partial failure (smaller than 1, or much lower than p). Parallelism has a direct influence on the construction of the B & B tree which is not the same as in sequential processing: sequential and parallel B & B are necessarily different.

These behavioural anomalies, both positive and negative, may at first seem surprising as they contradict expected results. The choice of relative instead of absolute speed-up helps to explain the occurrence of these anomalies which is due to the properties of the tree to be explored, of the priorities, and of the bounding function. These properties may only be recognized a posteriori once the exploration is completed. These have been the subject of a great deal of research since 1980 (we would like to cite [50], [51], [55] and refer to [93] for the others).

The time taken for a sequential B & B is related to the number of nodes in the fully-developed tree. The size of the tree in a B & B, where the branching principle, the bounding function v, and the priority h have been defined a priori (prior to execution), will depend on the search strategy used and the properties of v. In fact, the tree to be explored in a B & B is compound of four types of nodes:

– *critical* nodes C with a value strictly smaller than the optimal solution f^*;
– *undecidable* nodes M non-terminal (non-feasible solutions) with a value equal to f^*;
– *optimal* nodes O with value of f^*;
– *eliminated* nodes E with a value strictly greater than f^*.

In Fig. 1, $O = \{M, L\}$, $M = \{K, G, J\}$, $C = \{A, B, C, D, E, F, H, I\}$; the elements of E are not indicated.

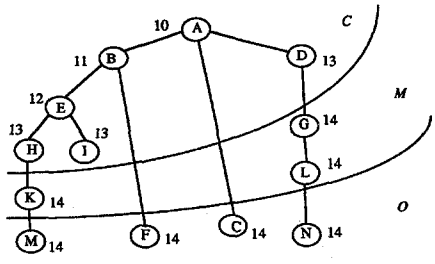

Fig. 1. B & B with best first strategy.

As a B & B is executed with a 'best first' strategy, it develops all the critical nodes, some undecidable nodes and one optimal node; certain nodes belonging to E can be explored with other strategies. Several executions may correspond to the same B & B according to the choices made by the strategy between nodes of equal priority, which may be very numerous (as shown in Table 2, where the first column indicates the total number explored in the B & B tree whereas the last one gives the number of nodes which have no distinct priority).

We define the minimal tree as the tree, regardless of the exploration strategy, which must be built to prove the optimality of a feasible solution and which has the minimal number of nodes. 'Best first' strategy contructs the minimal tree if there are no undecidable nodes ($M = \emptyset$), and if the bounding function is *discriminating*, which means that v is such that no nodes have equal priority.

In parallel processing, p processors must explore all the nodes in the minimal tree. In sequential processing, speed-up is always linear (lower than or equal to p) if the minimal tree has been built, i.e. if the sequential time is that of the 'best' possible sequential algorithm.

The fact that it is not always possible to define a priori the search strategy to construct the minimal tree shows that speed-ups may be favorable (the sequential tree is very large) or unfavorable, and therefore proves the existence of these anomalies. It is interesting to note that parallelism may have a corrective effect in sequential cases where the minimal tree has not been built.

In a theoretical synchronous context (where one iteration corresponds to the exploration of p nodes in parallel by p processors between two synchronizations), suitable conditions for avoiding detrimental anomalies if h is discriminating [55]. One necessary condition for the existence of favorable anomalies is that h should not be completely *consistent* with v (strictly higher priority means a lower or equal value). This is the case of the breadth or depth first strategy.

The best first strategy has proved to be very *robust* [94]. The speed-up it produces varies within a small interval around p. It avoids detrimental anomalies if v is discriminating, as we have already seen, or if it is *consistent* (at least one node of the sequential tree explored at each iteration) [62].

We have studied [62] rules for selecting between equal priority nodes, with the aim of avoiding unfavorable anomalies in 'best first' B & B's, thus eliminating processing overheads or memory occupation, which is not the case with the other proposals [98]. Three policies, based on the order in which nodes are created in the tree, have been compared: newest, oldest, and leftest. Bounds calculated on expected speed ups show that the 'oldest' rule is the least likely to produce anomalies.

It is therefore interesting to study the data structures which, unlike the well-known heaps, are capable of generating nodes with the same priority simultaneously (see Section 3.2).

As similar results (condition of existence) may be demonstrated when the number of processors is increased (growth anomalies or non-monotonous increases in speed up), some researchers are working on defining a measurement, the isoefficiency function [49], based on general research into the 'scability' of parallel algorithms: in order to maintain an efficiency of E with N processors, the size of the problem processed should grow according to this function $f(N)$.

We would like to point out that similar research (about bounds on speed-up) has been carried out into game tree traversal (critical tree, number of critical nodes,...; see a number of references in [47] and [22]).

3.2. Data structures

In sequential B & B, the choice of a suitable priority queue allows to save time when selecting and exploring nodes. This choice becomes crucial in parallel processing as we will demonstrate below.

In distributed parallelization, it enables each process to manage its local list. In centralized parallelization, it is absolutely crucial and highlights the problem of access to shared data (work): several processes may wish to execute an exploration (branching and bounding) at the same time on a B & B node stored in the priority queue managed by the central process. Consistency may then only be ensured in the shared priority queue by mutually exclusive access of the processes, which 're-sequentializes' this part of the algorithm. This causes a management overhead which limits the potential speed up due to parallelization. The quicker the operations are carried out on the priority queue (the better their theoretical complexity), the lower the overhead.

A first, rather simple, solution consists in increasing the work resource to be shared by dividing the exploration of a node into two sub-tasks, thus creating two priority queues: the first with the nodes to be separated, the second with the nodes to be evaluated [79]. Processing a node to be separated (to be evaluated) causes insertions and deletions of nodes in the other queue, according to B & B rules. The advantage is that the number of possible simultaneous accesses to work (a node to be processed) for inactive processes [79] is increased and a process which finds one priority queue busy may always try to access the other queue. 'Branching' and 'Bounding' tasks should still have a sufficiently coarse grain to avoid too many access conflicts.

Two queues including one with concurrent access may also be used for access to nodes in a game tree (α-β parallel method, [21]). Another way of increasing accessible resources is to create several work pools, associated to a subset of processors, where processes pick and store their units of work; the algorithm can be viewed as a centralized algorithm but with several central processes [27]. Let us also mention the splitting of the search interval S of the B & B into intervals which are each managed by one heap [99].

A final solution, on shared memory machines, consists of using priority queues which authorize simultaneous access and concurrent operations [86,36,24,64,53]. We would like to present more extensively one data structure that we have proposed, because it is a very simple priority queue and very easy to implement. It is highly suitable for sequential as well for parallel B & B. In the former case it is a funnel table while it is a funnel tree in the latter [64].

Funnel table or tree

This data structure uses the fact that stored priorities are in a small given interval $[1,\ldots,S]$. In most combinatorial optimization problems, the size S of the initial search interval, i.e. the difference between initial values of the lower bound, ilb, and the incumbent, iub, is small in relation to the total number of nodes in the tree, n_{tot}, and that of the active nodes in a given iteration, n_{max}. Table 2 illustrates the importance of the number of nodes with equal values, n_{equal}: in the case of the Travelling Salesman example, the size of the exploring interval S is 490, and as the number of nodes explored is 1558, at least 1068 nodes must have an evaluation equal to the evaluation of an other node (nodes with 'equal priority').

A simple table of dimension S is sufficient to ensure the *insert* and *deletegreater* operations in O(1) whereas the 'lowest' priority element, *deletemin*, is searched in the worst case in O(S) (starting from a pointer on the last lowest priority

Table 2
Nodes with equal priorities n_{equal}

Problems (precise reference in [64])	n_{tot}	n_{max}	ilb	iub	S = iub − ilb	$S' = 2^k$	n_{equal}
Quadratic Assignment	5054	2766	493	578	85	128	4969
Roucairol	11282	≥ 20000	963	1150	187	256	11095
Travelling Salesman	1558	"	7125	7615	490	512	1068
Padberg							
Jobshop	17982	"	808	969	161	256	17821
Carlier–Pinson	5012	"	5223	5484	261	512	4751

element found, moving this pointer each time the current element is empty). In order to run concurrent operations, we have added to this table a binary tree playing the part of a funnel to fill it, which gives us the name funnel tree (Fig. 2). The dimension of the table has been enlarged to S', the first power of 2 greater than S, so that the number of leaves of the funnel tree becomes S'.

Each leaf in the tree corresponds to a priority in Table 2 and nodes with equal priority are kept in a queue. The *funnel tree* makes it possible to manage nodes with equal priority, which is fundamental for improving performance. Each node in the tree has a counter storing the number of nodes in the queues of the leaves on the subtree of which it is the root. *Insert* and *deletegreater* operations are carried out bottom-up from the leaf towards the root: they require the modification of the counters at the nodes along the path from this leaf to the root.

Deletemin is carried out top-down from the root following the path which passes by the leftmost node whose counter is not at zero (refer to [64] for further details). The funnel tree is capable of running all the operations of a B & B with a complexity in time of $O(\log S)$ and in space of $O(n_{max} + 4S - 1)$ instead of $O(\log n_{max})$ with a classical heap.

Concurrent operations and priority queues

The following conditions are required for concurrent basic operations in tree data structures:

– Basic operations must all be carried out from the root towards a leaf (several techniques make it possible to 'reverse' operations [53]).

– A partial locking method must be developed, so that only a part of the tree may be locked: when a process carries out a basic operation on the tree structure, i.e. following a path from the root to a leaf, it locks only the node on which it is operating and its child-nodes (*locking window* or *bubble*).

Other well-known structures such as *heaps* [86,36], *D-heap*, *Pairing-heap*, *Leftist-heap*, *Skew-heap* and *Splay-tree*, may be used. Speed-ups obtained on a Sequent Balance (shared memory multiprocessors) or KSR 1 (virtually-shared memory) demonstrate the usefulness of the funnel table or tree as compared to other tree priority queues [53].

Research into priority queues suitable for dynamic management of irregular data generated while the program is running is an important new aspect of search space exploration, not only in RO but also in AI, as is shown by the importance of structures used to explore game trees with the α-β method [21,22], or state graphs with the A* algorithm (concurrent 'treap' combining a priority queue and a search tree in the same structure [21]).

3.3. Communication overhead

How can we find a good compromise between processing time and communication time? The grain of the work and the subsequent choice of a centralized or distributed version have a direct influence on the frequency of communications and are adapted to the architecture of the chosen machine in function of our knowledge of the problem to be processed.

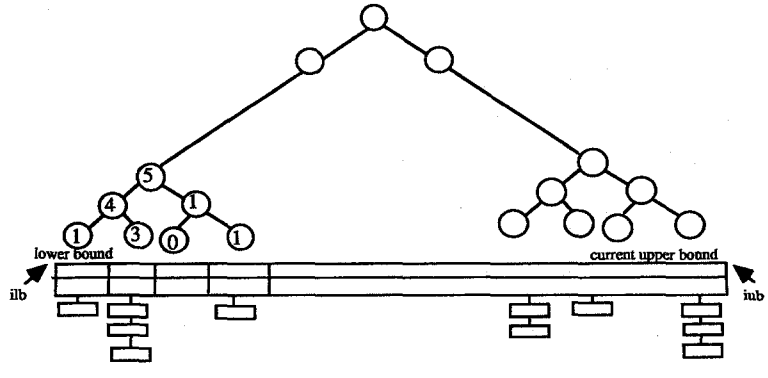

Fig. 2. Funnel tree.

Granularity

The work grain of a parallel algorithm is defined as the amount of work carried out by one processor independently of the others between two communications.

If the number of processors used is very large, speed-ups are restricted by the factor

$$(T_{acc} + T_{grain})/T_{acc},$$

where T_{acc} is work acquisition time and T_{grain} the amount of work between two communications. This shows the danger of using too fine a grain.

Work acquisition time T_{acc} may be divided into two parts: data access (data structure management, communication protocols) T_{data} and data transmission time T_{trans} intrinsic to the architecture of the machine.

It is possible to reduce the time T_{acc} by reducing T_{data} using efficient data structures and improving communication protocols. On the other hand, the only way of reducing T_{trans} is to improve the communication hardware, e.g. the network flow rate. Time T_{trans} is usually a fixed characteristic of the machine, and has a larger value in machines with physically distributed memories. It is therefore necessary to reduce the number of interprocessor communications as much as possible, particularly on this type of machine.

However, a total absence of communication and an increase in grain T_{grain} create a higher risk of redundant and useless work as each processor will only have access to information on its own load and will not known anything about the other processes.

3.4. Load balancing

In order to make full use of the processing capacity of parallel machines, the work must be shared as evenly as possible between the various processors. In a B & B algorithm, this may not be achieved in advance, as the whole tree has not yet been built, so work must be shared during the execution of the algorithm (*dynamic balancing*).

Even if a processor's local work is easily divisible into several independent parts, or nodes to be explored, the cost of dividing work and transferring it from one processor to another must not be excessive, i.e. greater than the cost of processing the work to be transferred. Furthermore, it is very difficult, if not impossible, for one processor to estimate the extent of all the other processors' work, due to lack of global information.

One of the special features of parallel B & B is that this dynamic sharing takes into account the 'quality' of work to be distributed: processes must work on nodes corresponding to equally 'promising' solutions (with similar values) to develop trees of a similar size. This necessitates a judicious initial allocation of work so that each processor has a sufficient quantity, and possibly requires the development of rules so that work migrates to processors which are not overloaded as it is created (for the sake of simplification, we assume that the problem of mapping processes on processors has been solved, and that each process is allocated to one processor).

Initial distribution

Solutions put forward for initial sharing consist of rapidly creating a sufficient number of nodes in the tree to be explored, i.e. a number greater than or equal to the number N of processors in the machine. This is carried out by developing the tree until it has N nodes:

– either with a single processor, usually by using the principle of branching in order to create more than two children-successors for the same node, or polytomic branching [91,20,94] (strongly-coupled architecture),

– or recursively with the same number of processors as there are nodes to be explored (weakly-coupled architecture).

Dynamic load balancing

A large number of heuristics obtained from probabilistic or non-probabilistic models are used to balance the load in various types of machine architectures (see a detailed bibliography on this subject [26,6]).

Strategies designed for B & B algorithms are based on the same principles. Work migrates:

– on the sender's initiative: a process declaring that it is overloaded according to its local load will send work to a neighbor;

– on the receiver's initiative: a process requests work when becoming underloaded.

The first strategy is best suited to slack periods, as in very busy periods processors may not be able to find any lightly-loaded processors. The second system is more advantageous when processors have very heavy loads, as there are few request messages. On the other hand, at light loads, this strategy is likely to saturate the communication channels.

A combined strategy is possible, in function of process load. However, it is likely to lead to unstable balancing, as work travels unnecessarily back and forth between processors. Thresholds triggering that changes of strategy may be adapted dynamically in this case.

In a *distributed-type parallelization*, the large grain of the work makes even sharing of construction of the tree between processes a crucial matter. Migration takes place:

(i) on the initiative of a process in function of the quantity of work it has at a given time (local rule);

(ii) as soon as the size of its local queue has increased or decreased by a given percentage since the last exchange. A process then sends either work (a predefined number of nodes from those with the best priority) or a work request [57,58] to a neighbor – another measurement used instead of the number of nodes stored takes their quality into account (the sum of the square of the size of the search interval for the nodes stored) [43];

(iii) when a process only replies to a work request from a neighbor if its local load is heavy and there is a significant difference with its neighbor's workload [57,58].

Some authors use a controller process which adjusts these thresholds during the algorithm according to the assumed variation in demand from other processes [58].

(iv) according to rules fixed in advance (nearest neighbor; interlaced or cyclical distribution of a node to each of the p processors [71,72]; regular or fixed intervals [75,18]);

(v) at random: nodes to be explored in a processor's local queue (or a feasible solution which may have been found) are sent at random to each (or to one) of the processors, depending on the situation [38,72]. The fact that this parallelization is synchronous makes it possible to carry out a probabilistic analysis (thanks to queuing theory and a model known as Shooting range [38]) and to prove theoretically, with an exponentially small probability, that the time will only exceed by a very low constant the time taken by any other strategy of load balancing;

(vi) by combining these various strategies: requesting work to a neighbor in case of inactivity, answering all requests if possible, and sending the best node to a neighbor at regular intervals [88]. The neighbor (or neighbors) to whom the work (or request) is sent depends mainly on the machine's communication network (ring, hypercube, shared memory, etc.) and on the algorithmic choice to restrict communications or not.

In the *centralized version*, balancing is simple: the centralized process distributes one node or a fixed number of nodes to each process. However, on shared memory machines, it may regulate access to the shared queue (allowed if the processed load of a process is no more than $t\%$ of the average processed load of the other processes [80]).

It may also check the quality of the nodes on which processes are working (blackboard technique [46]). If a process using the 'best first' strategy chooses a node in its local queue with a very low priority compared to the top priority of the nodes on the blackboard, some of the best nodes are transferred from the blackboard to the process' local queue. In the opposite extreme case (very high local priority), nodes are transferred in the other direction. The process only works on the chosen node if its priority is within a reasonable range around the priority of the best node on the blackboard.

In the case of a depth first strategy, we have defined access to the global work pool by applying the notion of *feeding tree* [63]. In the Quadratic Assignment Problem, the maximum depth of the B & B tree is equal to the size n of the problem: an element among the n elements is assigned at each level of the tree, thus n is the maximal depth. All inactive processes access the feeding tree in a mutually exclusive system and take a node at a depth i, a parameter of the application. It develops a local B & B tree rooted in this node and of maximum depth $(n - i)$. This application-dependent parameter i controls the grain and balances the processor load.

Rather than fully developing the search tree to

level i and let the pool of subproblems to be solved by the individual processors be the subproblems of level i, we propose that the 'search tree to level i' should be developped gradually by the processors as they need new work. The advantage is the saving of memory space, the drawback is that in a centralized environment, the process spends more time than necessary in the shared data structures.

The problem of task balancing is even more crucial but also more difficult on massively parallel SIMD machines. The load must be balanced globally. Unlike MIMD machines, where inactive processors may request work from a loaded processor without interrupting the work of the other processors, on an SIMD machine, all the other processors must suspend their activities momentarily. This leads to a highly paradoxical situation: load balancing inevitably reduces processor work rate, but it is necessary to balance the loads to achieve a good work rate. A B & B algorithm is executed on an SIMD machine in an alternating succession of exploration and load balancing phases. This additional difficulty, combined with the fact that local memories are usually small, means that, until now, very little B & B work has been implemented on this type of machine [23,81].

4. Applications

Historically, the first applications dealt with 'simulated' parallel machines, or experimental systems, often with outlandish architectures and other drawbacks due to their small size: Knapsack, Travelling Salesman, Puzzle, Gauss' Queens Problem (for proofs, refer to the state of the art in 1985 [39], 1987 [89] and 1988 [93]). Authors at that time were simply trying to test their parallelization methods, and their results, considerably inferior to those of sequential systems, did not give a convincing account of parallelism.

Since 1985, research into performances, data structures, and load balancing, validated both in theory and practice, has made it possible to deal more effectively with problems raised by the parallelization of tree exploration. It has become possible to experiment on commercial machines, multiprocessors with shared memory or networks and to deal with problems of a more realistic size.

To mention a few of the successfully-processed problems (linear speed-ups):

– *integer linear programming:* benchmark on standard problems using Hep, Sequent and Encore by Bœhing et al. [12], and experiment with the IBM OSL solver [17];

– *quadratic 0–1 programming* without constraints: results presented with a size-100 array, on the 4 processors of the Cray XMP 148, the 6 of the IBM 3090-600E (6), the 32 and 16 of the Intel iPSC/1 and /2 (Pardalos and Rodgers [74]);

– *linear 0–1 programming:* local network of 8 workstations DEC Vax (Cannon and Hoffman [15]);

– *mixed linear integer programming*, on the 128 processors of the CM-5: benchmark MIPLIB library problems, solving applications of considerable size with good speed-up and times on the order of a few seconds, in air transport, crew scheduling, warehouse location, optical network design, etc. (Eckstein [27]);

– *quadratic assignment:* problems up to size 12 solved in literature on IBM 3090-400E, (Crouse and Pardalos [20]); on Cray X-MP/48 (Roucairol [91]); and recently, the first exact solution to a size 18 Nugent problem on Cray 2 [63], to a size 20 on IBM SP1;

– *set covering:* on Intel iSPC 12 (Rushmeir and Nemhauser [97]);

– *minimal vertex cover:* graph with 150 nodes, and an average degree of 30, on a network of 64 Transputers (Lüling and Monien [57]); graph with 80 nodes on a BBN Butterfly, (Kumar et al. [46]); on 32 transputers (Vornberger [106]);

– *two-dimensional cutting stock:* Kröger and Vornberger [43];

– *workshop scheduling* on 16 Transputers FPS T-20: Pargas and Wooster [75];

– *knapsack:* results obtained by parallel processing are only competitive with sequential results in the case of the multiknapsack; problems in the literature in Plateau and Roucairol [80]);

– *multicommodity location problem* with balancing requirements: Gendron and Crainic [31];

– *AI problems* such as puzzle: up to the size 15, 4×4 checkerboard, on the Connection Machine (Powley, Ferguson and Korf [81]), on a BBN Butterfly (Kumar et al. [46]), on KSR 1 (Le Cun and Cung

[21]), mapping queens not in check on a 126×126 chessboard, knights on an 8×8, 6×6 magic square (on Sequent Symmetry, see Saletore and Kalé [98]);
– *game tree search:* on simulated trees, a large number of experiments on shared memory multiprocessors and networks (see Cung and Roucairol [21]);
– *problems arising in VLSI design:* indivisible block mapping, satisfiability, on N Cube/10, (Kumar and Rao [46]);
– *exploration of AND / OR graphs in logic programming:* refer mainly to work by Kumar and Kanal [45,48,56,etc.].
– and, last but not least, the *Travelling Salesman problem* (TSP).

Indeed, the best results in parallelism have been obtained with this problem. For an asymmetric problem (travelling cost between two towns depends on the direction), Pekny and Miller [67,79] found the optimal solution ('exact solution') to problems with 70 000 towns in approximately 20 minutes on a BBN Butterfly which may be considered to be a 'modest' parallel machine: its power of 20 mips with 10 processors is smaller than that of many sequential machines!

Problems with 1000 and 3000 towns in difficult configurations (making heuristic search methods ineffective) only took one and two minutes respectively! The fact that these problems can be solved efficiently in parallel makes it possible to solve various manufacturing problems, of which the asymmetric Travelling Salesman is a formal expression: no-wait flowshop scheduling [77], and scheduling with production costs dependent on the sequencing of consecutive tasks [78].

A first "Parallel Computing of Discrete Optimization Problems" workshop, organized in 1991 at the High Computing Center, at Minneapolis University, with the American Navy (FMC Corporation Naval Systems division), had already demonstrated the contribution of parallelism towards solving large size practical problems as well as those with economic repercussions: in chemical engineering (Du Pont de Nemours, asymmetric TSP on the Nectar heterogeneous workstation network, systolic table, multiprocessors, massively parallel SIMD machine by Pekny and Miller [67]), in airline management (American Air Lines, symmetric Travelling Salesman solved by Branch & Cut, up to 12.75 million variables, on various hypercubes from 32 to 64 nodes, by Bixby et al. [11]). The second workshop, "Parallel Processing of Discrete Optimization Problems", held at DIMACS, Rutgers University, in April 1994, the DIMACS Challenge on parallel algorithms, in October 94, and the recent Parallel Optimization Colloquium, POC '96, held at Versailles, have confirmed the major share of large applications and parallel B & B algorithms.

5. Prospects

We have certainly acquired a great deal of expertise on a number of parallel machines, spectacular results have been obtained with large size applications and for industrial problems (Mixed Programming MIP, Travelling Salesman TSP) but considerable efforts remain to be made to convince companies of all sizes to develop parallel programs.

In view of the difficulty of programming (implementation of a B & B algorithm on the one hand, choosing a parallelization system on the other hand), several research scientists have suggested programming environments dedicated to B & B algorithms [109,29,100,25] or have outlined the basic aspects of a generic B & B program ([65,25,42], for distributed B & B algorithm see [27] and [44]). Like all meta-algorithms, although it is less powerful than a custom-made algorithm, it facilitates the writing of B & B procedure for any application and makes it possible to test several parallelization parameters. We have proposed a B & B library BoB, which allows to test several data structures, several parallelization schemes, several search and load balancing strategies, and to develop portable programs which can operate on currently available massively parallel machines as well as on networks of workstations [8].

In view of the difficulty of acquiring or obtaining access to these machines, and also of choosing an architecture, current experiments are diversifying into using workstation networks [15,27,105]. Operating tools and message-passing libraries such as PVM (Parallel Virtual Machine [30]), by simulating virtual parallel machine architecture, make it possible to develop portable programs which can operate both on this type of network as well as on currently available massively parallel machines.

6. Parallelizing heuristic search methods

If the preceding methods are considered as search space exploration methods, meta-heuristics like simulated annealing (SA), tabu search (TS), or genetic algorithms (GA), may also be more or less included in this category. Search space potential may be represented by the graph of transitions between problem states (a feasible solution or not for SA and TS, a new generation of population for GA). Exploration consists of building a neighborhood, evaluating the states in the neighborhood and choosing one state to continue.

Having made this analogy (we refer to a more detailed presentation of these metaheuristics in the book by Reeves [89]), parallelization of these methods is centralized or distributed in the same way (see Aarts and Korst [1], Greening [34], Rudolph [96] and Roussel-Ragot [87] for SA, Crainic [19] and Voss [107] for TS, and Mühlenbein [69,70] for GA).

In the centralized version, the central process executes the algorithm by delegating work to other processes: exploring the neighborhood, evaluating the solutions found. The rather fine grain of the work and the fact that exploration is deliberately highly controlled mean that this version is synchronous most of the time. The central process collects the information, chooses a solution and redistributes the work, which is known in advance, in an equitable way. The starting point for a different exploration by the processes is either this solution, or the solutions which may be reached from this solution by one movement.

This synchronization is triggered after one movement, as soon as one (or more) of the reachable solutions allocated to each process have all been evaluated (TS: [4], [101] and [16]) or after a fixed number of iterations, leaving each process responsible for developing its own exploration (TS: [59] and [102]). In the asynchronous version, the best solution is updated by the central process which accepts the movement proposed by a process [2].

In the distributed version, each process organizes its own search and establishes communication with the others on its own initiative, to communicate a best solution (according to local or global criteria) sent to all the processes.

There is a strong resemblance between the parallelization of B & B algorithms and the parallelizations of heuristic searchs. But, whereas there is a tremendous number of experiments with heuristic methods on sequential machines, those on parallel machines concerning combinatorial optimization problems are fairly recent and not very numerous.

We would like to mention:

– for Simulated Annealing, school time tabling by Abramson [2], and the Travelling Salesman Problem by Malek et al. [59], Bertocchi [9] and Allwright et al. [3]:

– for Tabu Search, the inevitable Travelling Salesman Problem (Malek et al. [59]); on the 10 processors of the Sequent Balance 8000, on 4 simulated processors, Vehicle Routing by Taillard [102]; very large size Quadratic Assignment (allocation of up to 80 elements) on a ring of 10 Transputers by Taillard [101], and on one Connection machine CM-2, Chakrapani and Skorin-Kapov [16] and August and Mautor [4]; and multicommodity location with balancing requirements (Crainic et al. [19]) on a network of 8 SUN Sparc;

– for genetic algorithms, Quadratic Assignment (Brown et al. [13]); process mapping on processors (Talbi and Bessière [103]); and Flowshop (Bierwirth et al. [10]). It is clear that these experiments are just beginning and should develop in the future, especially asynchronous applications. What is lacking at the moment is the fundamental research which would enable us to understand, as in the case of parallelization of B & B algorithm, how parallelism influences and modifies the exploration.

7. Conclusion

If parallelism is used properly in search space exploration of combinatorial optimization problems (suitable data structure, efficient load balancing strategies) it helps to restrict the combinatorial explosion and to solve problems efficiently (linear speed up) and accurately (obtaining optimal solutions) for practical cases of considerable size. The performances which have been obtained until now for these problems, which are mostly NP-complete, are in direct contradiction with arguments put forward by people who make the generalization that

"solving NP-complete problems is synonymous with heuristic search methods"!

For large-scale problems where heuristic search methods are still the only viable ones, parallelism may accelerate convergence towards a good local optimum as compared to sequential processing.

Nevertheless, in view of the very different characteristics of currently marketed machines (shared memory multiprocessors, distributed networks, heterogeneous networks, and massively parallel SIMD machines) which are bound to become more widely used in the future, we must continue to increase the number of experiments, create libraries of parallel programs, and design portable algorithms which may also be used on workstations organized as virtual parallel machines.

References

[1] Aarts, E.H.L. and Korst, J.H.M., *Simulated Annealing and Boltzmann Machines*, Wiley, Chichester, 1989.

[2] Abramson, D., "Constructing school time tables using simulated annealing: Sequential and parallel algorithms", *Management Science* 37 (1991) 98–113.

[3] Allwright, J.A., and Carpenter, D.B., "A distributed implementation of simulated annealing for the travelling salesman problem", *Parallel Computing* 10 (1989) 335–338.

[4] Arvindam, S., Kumar, V., and Rao, U.N., "Efficient parallel algorithms for search problems: Applications in VLSI CAD", in: *Proc. 3rd Symp. of Massively parallel computation*, 1990.

[5] August, N., and Mautor, T., "Méthodes de recherche tabou massivement parallèle pour le problème d'affectation quadratique", Tech. Rep. INRIA 2182, Rocquencourt, France, 1994.

[6] Authié, G., Ferreira, A., Roch, J.L., Villard, G., Roman, J., Roucairol, C., and Virot, B., *Algorithmes Parallèles: Analyse et Conception*, Hermès, France, 1994.

[7] Beasley, J.E., "Supercomputers and OR", *Journal of the Operations Research Society* 38/11 (1987) 1085–1089.

[8] Benaïchouche, M., Dowaji, S., Le Cun, B., Mautor, T., Roucairol, C., "BoB: A unified platform for implementing Branch and Bound like algorithms", Tech. Rep., Univ. Versailles, PRiSM 95/16 Paris, France, 1995.

[9] Bertocchi, M., and Sergi, P., "Parallel global optimization over continuous domain by simulated annealing", in: P. Messina and A Murli (eds.), *Proceedings of Parallel computing: Problems, Methods and Applications*, Elsevier Science Publishers, Amsterdam, 1992, 87–97.

[10] Bierwirth, C., and Stöppler, S., "The application of parallel genetic algorithm to the $n/m/P/$Cmax flowshop problem", in: G. Fandel, Th. Gulledge and A. Jones (eds.) *New Directions for Operations Research in Manufacturing*, Springer, Berlin, 1992.

[11] Bixby, R., "Two applications in linear programming", in: *Proc. of Workshop on Parallel Computing of Discrete Optimization Problems*, Univ. Minneapolis-AHPC, 1991.

[12] Bœhning, R.L., Butler, R.M., and Gillett, B.E., "A parallel integer linear programming algorithm", *European Journal of Operational Research* 34 (1988) 393–398.

[13] Brown, D., Huntley, C., and Spillane, A., "A parallel genetic heuristic for the quadratic assignment problem", in: *Proc. on conference on genetic algorithms*, Arlington, VA, 1989, 406–415.

[14] Burton, F.W., McKeown, G.P., Rayward-Smith, V.J., and Sleep, M.R., "Parallel processing and combinatorial optimization", in: L.B. Wilson, C.S. Edwards and V.J. Rayward-Smith (eds.), *Combinatorial Optimization III*, Univ. of Stirling, UK, 1982, 19–36.

[15] Cannon, T.L., and Hoffman, K.L., "Large-scale 0–1 Linear Programming on distributed workstations", *Annals of Operations Research* 22 (1990) 181–217.

[16] Chakrapani, J., and Skorin-Karpov, J., "Massively parallel tabu search for the quadratic assignment problem", *Annals of Operations Research* 41 (1993) 327–341.

[17] Ciriani, T., "A Branch and Bound library", ECCO VI, European Chapter on Combinatorial Optimization, Brussels, 1993.

[18] Clausen, J., and Träff, J.L., "Implementation of parallel branch and bound algorithms: Experiences with the graph partitioning problem", *Annals of Operations Research* 33 (1991) 331–349.

[19] Crainic, T.G., Toulouse, M., and Gendreau, M., "Towards a taxonomy of parallel tabu search algorithms", Tech. Rep. CRT-933, Centre de Recherche sur les transports, Université de Montréal, Canada, 1993.

[20] Crouse, J., and Pardalos, P., "A parallel algorithm for the quadratic assignment problem", in: *Proc. of Supercomputing 89*, ACM, 1989, 351–360.

[21] Cung, V.-D., and Roucairol, C., "Parcours parallèle d'arbres minimax", Tech. Rep. INRIA 1549, Rocquencourt, France, 1991.

[22] Cung, V.D., "Contribution à l'algorithmique non numérique parallèle: Exploration d'espaces de recherche", Thèse Univ. Paris 6, Paris, France, 1994.

[23] Dehne, F., Ferreira, A., and Rau Chaplin, A., "Parallel Branch and Bound on fine grained hypercubes multiprocessors", *Parallel Computing* 15/1–3 (1990) 201–209.

[24] Deo, N., "Data structures for parallel computation on shared memory machines", in: J.S. Kowalik (ed.), *Supercomputing 89*, NATO ASI Series Vol. F62, Springer, Berlin, 1990, 341–345.

[25] Diamond, M.J., Kimbel, C., Rennolet, C.L., and Ross, S.E., "PICO: Parallel implementation of Combinatorial Optimization", Workshop on Parallel Computing of Discrete Optimization Problems, Univ. Minneapolis-AHPC, 1991.

[26] Dowadji, S., "Equilibrage des tâches: Un état de l'art", Tech. Rep., Univ. Paris 6, MASI 101, Paris, 1992.

[27] Eckstein, J., "Parallel Branch and Bound algorithms for

general mixed integer programming on the CM 5", Tech. Rep., Thinking Machine Corporation TMC-257, 1993.

[28] El Dessouki, O., and Huen, W.H., "Distributed enumeration on networks computers", *IEEE Transactions on Computers* 29 (1980) 818–825.

[29] Finkel, R., and Manber, U., "DIB – A distributed implementation of backtracking", *ACM Transactions on Programming Language Systems* 9 (1987) 235–256.

[30] Geist, A., et al., "PVM 3.0 User guide and reference manual", Tech. Rep. No. ORNL/TM-12187, Oak Ridge National Laboratory, 1993.

[31] Gendron, B., and Crainic, T.C., "Parallel implementations of a Branch and Bound algorithm for Multicommodity location with balancing requirements", Tech. Rep. CRT-813, CRT, Univ. Montréal, Canada, 1992.

[32] Gendron, B., and Crainic, T.C., "Parallel Branch and Bound algorithms: A survey and synthesis", *Operations Research* 42, 1042–1066.

[33] Gengler, M., and Coray, G., "A parallel best first B & B with synchronization phases, in: L. Bougé, M. Cosnard, Y. Robert, D. Trystram (eds.), *Proc. of CONPAR 92*, Series, LNCS 634, Springer, Berlin, 1992, 515–526.

[34] Greening, D.R., "Parallel simulated annealing techniques", *Physica D* 42 (1990) 293–306.

[35] Imaï, M., and Fukuruma, T., "A parallelized Branch and Bound algorithms implementation and efficiency", *Systems Computers Controls* 10/3 (1979) 62–70.

[36] Jones, D.W., "Concurrent operations on priority queues", *Communications of the ACM* 32/1 (1989) 132–137.

[37] Karp, R.M., and Ramachandran, V., "A survey of parallel algorithms for shared memory machines", Tech. Rep UCB/CSD 88/408, Univ. of California, Berkeley, CA, 1988.

[38] Karp, R.M., and Zhang, Y., "A randomized parallel Branch and Bound procedure", in: *Proc. of ACM Symposium on Theory of Computing*, 1988, 290–300.

[39] Kindervater, G.A.P., and Lenstra, J.K., "Parallel algorithms", in: M. O'hEigeartaigh, J.K. Lenstra and A.H.G. Rinnooy Kan (eds.), *Combinatorial Optimization: Annoted Bibliographies*, Wiley, New York, 1985, 106–128.

[40] Kindervater, G.A.P., and Lenstra, J.K., "Parallel computing in combinatorial optimization", *Annals of Operations Research* 14 (1988) 245–289.

[41] Kindervater, G.A.P., Lenstra, J.K., and Rinnooy Kan, A.H.G., "Perspectives on parallel computing", *Operations Research* 37/6 (1989) 985–990.

[42] Kindervater, G.A.P., "A virtual parallel B & B machine", Workshop Parallel Processing of Discrete Optimization Problems, Dimacs center, Rutgers, New Brunswick, 1994.

[43] Kröger, B., and Vornberger, O.,"A parallel Branch and Bound approach for solving a two dimensional cutting stock problem"; Tech. Rep., Dept. of Math. and computer Sc., Univ. of Osnabruck, Germany, 1990.

[44] Kudva, G.K., and Pekny, J.F., "DCABB: A distributed control architecture for B & B computations", ORSA/TIMS Conference, Phoenix, AZ, 1993.

[45] Kumar, V., and Kanal, L., "Parallel Branch and Bound formulations for AND/OR tree search", *IEEE Transactions on Pattern Analysis and Machine Intelligence* 6/6 (1984) 768–788.

[46] Kumar, V., Ramesh, K., and Rao, V.N., "Parallel best-first search of state-space graphs: A summary of results", in: *Proc. of the Nat. Conf. on Artificial Intelligence*, 1988, 122–126.

[47] Kumar, V., Gopalakrishnan, P.S., and Kanal, L., *Parallel Algorithms in Machine Intelligence and Vision*, Springer, Berlin, 1990.

[48] Kumar, V., and Rao, V.N., "Parallel depth-first search, part II: Analysis", *International Journal of Parallel Programming* 16/6 (1992) 501–519.

[49] Kumar, V., and Gupta, A., "Analyzing the scability of parallel algorithms and architectures", Inter. Conf. on Supercomputing, 1991.

[50] Lai, T.H., and Sahni, S.,"Anomalies in parallel Branch and Bound algorithms", *Communications of the ACM* 27 (1984) 594–602.

[51] Lai, T.H., and Sprague, A., "Performance of parallel Branch and Bound algorithms", *IEEE Transactions on Computers* 34 (1985) 962–964.

[52] Lavallée, I. and Roucairol, C., "A parallel B & B algorithm", Tech. Rep. LRI No. 164, Univ. Orsay, 1984.

[53] Le Cun, B., Mans, B., and Roucairol, C., "Opérations concurrentes et files de priorité", Tech. Rep. INRIA No. 1548, 1991.

[54] Le Cun, B., and Cung, V.D., "An efficient implementation of parallel A*", in: *Proceedings of Canada-France Conference on Parallel Computing*, Montreal, 1994.

[55] Li, G.J., and Wah, B.W., "Computational efficiency of parallel approximate Branch and Bound algorithms", in: *Proc. Inter. Conf. on Parallel Processing*, 1984, 473–480.

[56] Lin, Y., and Kumar, V., "An execution model for exploiting AND-parallelism in logic programs", *New Generation Computing* 5/4 (1988) 393–425.

[57] Lüling, R., and Monien, B., *Two Strategies for Solving the Vertex Cover Problem on a Transputer Network*, Lecture Notes in Computer Science 392, Springer, Berlin, 1989, 160–170.

[58] Lüling, R., and Monien, B., "Load balancing for distributed Branch and Bound algorithms", Tech. Rep., Dept of Math. and computer Sc., Univ. of Paderborn, Germany, 1991.

[59] Malek, M., Guruswamy, M., Pandya, M., and Owens, H., "Serial and parallel simulated annealing and tabu search algorithms for the Travelling Salesman Problem", *Annals of Operations Research* 21 (1989) 59–84.

[60] Mans, B., "Contribution à l'algorithmique parallèle: Parallélisation des méthodes de recherche arborescente", Ph.D. Thesis, Université Paris VI, France, 1992.

[61] Mans, B., Mautor, T., and Roucairol, C., "Recent exact and approximate algorithms for the quadratic assignment problem", in: *Proc. of Symposium on Applied Mathematical Programming and modeling*, APMOD 93, Budapest, 1993, 395–402.

[62] Mans, B., and Roucairol, C., "Performance of parallel Branch and Bound algorithms with best first search", *Discrete Applied Mathematics* 66 (1996) 57–74.

[63] Mans, B., Mautor, T., and Roucairol, C., "A parallel depth first search branch and bound algorithm for the quadratic assignment problem", *European Journal of Operational Research* 81 (1995) 617–628.

[64] Mans, B., and Roucairol, C., "Characterization of data structures for parallel Branch and Bound algorithms", in: *Proceedings of the European Congress of Operational Research*, EURO'XI, Aix-la-Chapelle, Allemagne, 1991.

[65] McKeown, G.P., Rayward-Smith, V.J., and Turpin, H.J., "Branch and Bound as a higher order function", *Annals of Operations Research* 33 (1991) 379–402.

[66] Miller, D.L., and Pekny, J.F., "Exact solution of larger asymmetric travelling salesman problems", *Science* 251 (1991) 754–761.

[67] Miller, D.L., and Pekny, J.F., "Exact distributed algorithms for travelling salesman problems", in: *Proc. of Workshop on Parallel Computing of Discrete Optimization Problems*, Univ. Minneapolis-AHPC, 1991.

[68] Mohan, J., "Experience with two parallel programs solving the travelling salesman problem", IEEE Int. Conf. on parallel processing, 1983, 191–193.

[69] Mühlenbein, H., Gorges-Scheutler, M., and Krämer, O., "Evolution algorithm in combinatorial optimization", *Parallel Computing* 7 (1988) 65–85.

[70] Mühlenbein, H., "Evolution in time and space – The parallel genetic algorithm", in: R. Rawlins (ed.), *Foundations of Genetic Algorithms*, Morgan Kaufmann, 1991, San Mateo, CA, 316–337.

[71] Ortega, M., and Troya, J., "Live nodes distributions in parallel Branch and Bound algorithms", *Microprocessing and Microprogramming* 25, (1989) 301–306.

[72] Ortega, M., and Troya, J., "A study of parallel Branch and Bound algorithms with best first search", *Parallel Computing* 11 (1989) 121–126.

[73] Papadimitriou, C.H., and Steiglitz, K., *Combinatorial Optimization: Algorithms and Complexity*, Prentice-Hall, Englewood Cliffs, NY, 1982.

[74] Pardalos, P.M., and Rodgers, G.P., "Parallel Branch and Bound algorithms for unconstrained quadratic zero–one programming", in: R. Shandra, B.L. Golden, E. Wasil, O. Balci and W. Stewart (eds.), *Impact of Recent Computer Advances in Operational Research*, Elsevier Science Publishers, North-Holland, 1989.

[75] Pargas, R.P., and Wooster, E.D., "Branch and Bound algorithms on a hypercube", in: *Proc. of Conf. on Hypercube Concurrent Computers and Applications, Vol. II*, 1988, 1514–1519.

[76] Pearl, J., *Heuristics, Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, Reading, MA, 1984.

[77] Pekny, J.F., Miller, D.L., and McRae, G.J., "Application of a parallel Travelling Salesman problem: Algorithm to no-wait flowshop scheduling", Engineering Design Research Center Tech. Rep. 06-51-89, Carnegie Mellon Univ., Pittsburg, 1989.

[78] Pekny, J.F., Miller, D.L., and McRae, G.J., "An exact parallel algorithm for scheduling when production costs depend on consecutive system states", Engineering Design Research Center Tech. Rep. 06-52-89, Carnegie Mellon Univ., Pittsburg, 1989.

[79] Pekny, J.F., and Miller, D.L., "A parallel Branch and Bound algorithm for solving large asymmetric travelling salesman problems", *Mathematical Programming* 55 (1992) 17–33.

[80] Plateau, G., and Roucairol, C., "A supercomputer algorithms for the 0–1 multiknapsack problem", in: R. Shandra, B.L. Golden, E. Wasil, O. Balci and W. Stewart (eds.), *Impact of Recent Computers Advances on Operational Research*, Elsevier Science Publishers, North-Holland, 1989, 144–157.

[81] Powley, C., Ferguson, C., and Korf, R.E., "Parallel tree search on a SIMD Machine", in: *Proc. of the 3rd IEEE Symp. on Parallel and Distributed Processing*, Dallas, TX, 1991.

[82] Pruul, E., "Parallel processing and a Branch and bound algorithm", M.Sc. Thesis, School of Operations Research and Industrial Engineering, Cornell Univ., Ithaca, NY, 1975.

[83] Pruul, E., Nemhauser, G.L., and Rushmeier, R.A., "Branch and Bound and parallel computation: A historical note", *Operations Research Letters* 7/2 (1988) 65–69.

[84] Quinn, M.J., "Implementing best first Branch and Bound algorithms on hypercube multicomputers", in: M. Health (ed.), *Hypercube Multiprocessors*, SIAM Press, Philadelphia, PA, 1987, 318–326.

[85] Quinn, M.J., *Designing Efficient Algorithms for Parallel Computers*, McGraw-Hill, New York, 1987.

[86] Rao, V.N., and Kumar, V., "Concurrent insertions and deletions in a priority queuee", in: *IEEE Proc. of Int. Conf. on Parallel Processing*, 1988, 207–211.

[87] Roussel-Ragot, P., "La méthode du recuit simulé: Accélération et parallélisation", Thèse de l'Université Paris 6, Paris, France, 1990.

[88] Rayward-Smith, V.J., Rush, S.A., and McKeown, G.P., "Efficiency consideration in the implementation of parallel Branch and Bound", Tech. Rep., School of Information Systems, Univ. of East Anglia, Norwich, UK, 1991.

[89] Reeves, C.R., *Modern Heuristic Techniques for Combinatorial Problems*, Advanced Topics in Computer Science Series, Blackwell Scientific Publications, Oxford, 1993.

[90] Ribeiro, C., "Parallel computer models and combinatorial algorithms", *Annals of Discrete Mathematics* 31 (1987) 325–364.

[91] Roucairol, C., "A parallel Branch and Bound algorithm for the quadratic assignment problem", *Discrete Applied Mathematics* 18 (1987) 211–225.

[92] Roucairol, C., "Parallel computing in combinatorial optimization", *Computer Physics Reports* 11 (1989) 195–220.

[93] Roucairol, C., "Parallel Branch and Bound algorithms: An overview", in: M. Cosnard, Y. Robert, P. Quinton and M.

Raynal (eds.), *Parallel and Distributed Algorithms*, Elsevier Science Publishers, North-Holland, 1988, 153–163.

[94] Roucairol, C., "Recherche arborescente en parallèle", Tech. Rep. MASI 90.4, Université Paris 6, Paris, France, 1990.

[95] Roucairol, C., "Parallel Branch and Bound on shared memory multiprocessors", Workshop on Parallel Computing of Discrete Optimization Problems, Univ. Minneapolis-AHPC, 1991.

[96] Rudolph, G., "Parallel simulated annealing and its relation to evolutionary algorithms", in: *Proc. of Symposium on Applied Mathematical Programming and Modeling*, APMOD 93, Budapest, 1993, 508–515.

[97] Rushmeier, M., and Nemhauser, G., "Performances of a parallel B & B for the set covering problem", Tech. Rep. J 89-02, School of Ind. Eng., Georgia Inst. of Technology, 1989.

[98] Saletore, V.A., and Kalé, L.V., "Consistent linear speedups to a first solution in parallel state-space search", in: *Proc. of the Nat. Conf. on Artificial Intelligence*, 1988, 227–233.

[99] Sarkar, U.K., Chakrabarti, P.P., Ghose, S., and De Sarkar, S.C., "Multiple stack Branch and Bound", *Information Processing Letters* 37 (1991) 43–48.

[100] Wei, S., and Kalé, L.V., "A dynamic scheduling strategy for the chare kernel system", in: *Proc. Supercomputing 89*, 1989, 389–398.

[101] Taillard, E., "Robust taboo search for the quadratic assignment problem", *Parallel Computing* 17 (1991) 443–455.

[102] Taillard, E., "Parallel iterative search methods for vehicule routing problems", Tech. Rep. ORWP 92/03, Dept. of Math., Ecole Polytechnique Fédérale de Lausanne, Switzerland, 1992.

[103] Talbi, E.G., and Bessière, P., "A parallel genetic algorithm applied to the mapping problem", in: *SIAM News*, 1991, 12–27.

[104] Trienekens, H.W.J.M., "Computational experiments with an asynchronous parallel Branch and Bound algorithm", Tech. Rep. EUR-CS-89-02, Computer Science Dept., Fac. of Economics, Erasmus Univ., Rotterdam, 1989.

[105] Tschöke, S., "A portable parallel Branch and Bound library", ECCO VII, European Chapter on Combinatorial Optimization, Milano, 1994.

[106] Vornberger, O., "Load balancing in a network of transputers", Second Workshop on Distributed Algorithms, Lecture Notes in Computer Science 312, 1987, 116–126.

[107] Voss, S., "Concepts for parallel tabu search", in: *Proc. of Symposium on Applied Mathematical Programming and Modeling*, APMOD 93, Budapest, 1993, 595–604.

[108] Wah, B.W., Li, G.J., and Yu, C.F., "Multiprocessing of combinatorial search problems", *IEEE Computers* (1985) 93–108.

[109] Wah, B.W., and Ma, Y.W., "MANIP – A multicomputer architecture for solving combinatorial extremum search problems", *IEEE Transactions on Computers* 33/5 (1984) 377–390.